

# COURSE ASSIGNMENT REPORT

## Web Automation & REST API Testing using Selenium WebDriver with Python

*Wipro Pre-Placement Python Automation Program*

---

### Student Submission Details

**Student Name:** Tanisha Pan  
**Enrolment No.:** 12023002029111  
**Stream:** Computer Science and Engineering (IoT, CS & BT)  
**Campus:** IEM Newtown  
**Assignment No.:** Assignment 2  
**Module:** Data-Driven UI Automation & REST API BDD Validations

Academic Year 2023–2027

# Contents

|          |                                                      |          |
|----------|------------------------------------------------------|----------|
| <b>1</b> | <b>Introduction &amp; Objectives</b>                 | <b>2</b> |
| 1.1      | Core Objectives . . . . .                            | 2        |
| 1.2      | Tools & Technology Stack . . . . .                   | 2        |
| <b>2</b> | <b>Target Applications &amp; Endpoints</b>           | <b>3</b> |
| 2.1      | Web UI Target . . . . .                              | 3        |
| 2.2      | REST API Target . . . . .                            | 3        |
| <b>3</b> | <b>Test Architecture &amp; Design</b>                | <b>3</b> |
| <b>4</b> | <b>Implementation Details &amp; Key Code</b>         | <b>3</b> |
| 4.1      | Dynamic Driver Resolution & Factory . . . . .        | 3        |
| 4.2      | Data-Driven UI Login Suite . . . . .                 | 4        |
| 4.3      | REST API BDD Validation Suite . . . . .              | 5        |
| <b>5</b> | <b>Test Scenarios &amp; Dataset Matrix</b>           | <b>5</b> |
| 5.1      | UI Data-Driven Test Dataset . . . . .                | 5        |
| 5.2      | REST API Scenario Matrix . . . . .                   | 6        |
| <b>6</b> | <b>Console Output &amp; Execution Logs</b>           | <b>6</b> |
| <b>7</b> | <b>Test Results &amp; Verdict Summary</b>            | <b>7</b> |
| 7.1      | Execution Metrics . . . . .                          | 7        |
| 7.2      | Detailed Result Assessment . . . . .                 | 7        |
| <b>8</b> | <b>Key Technical Highlights &amp; Best Practices</b> | <b>8</b> |
| <b>9</b> | <b>Conclusion</b>                                    | <b>8</b> |

# 1 Introduction & Objectives

This technical laboratory report documents the implementation, execution, and validation of **Assignment 2** completed as part of the Wipro Pre-Placement Python Automation Program. The curriculum targets industry-standard test automation proficiencies, specifically bridging **Data-Driven User Interface (UI) functional validation** and **RESTful API integration testing** under Behavior-Driven Development (BDD) semantic conventions (GIVEN, WHEN, THEN, AND).

## 1.1 Core Objectives

- **Data-Driven UI Automation:** Design an end-to-end parameter-driven testing harness to validate authentication behaviors across valid, invalid, and edge-case credential sets without cross-test state pollution.
- **Explicit Dynamic Synchronization:** Completely eliminate brittle, fixed-delay sleeps (`time.sleep`) by integrating `WebDriverWait` paired with explicit `expected_conditions` (`visibility_of_element_located`, `element_to_be_clickable`).
- **Headless & Resilient Driver Management:** Implement an adaptive driver factory capable of discovering local system binaries (e.g., GeckoDriver in Linux/POSIX environments) with headless flag overrides.
- **Automated Defect Telemetry:** Construct exception handlers that capture timestamped browser viewport screenshots upon assertion or locator timeout failures.
- **REST API Functional Validation:** Perform automated HTTP GET and POST requests using the `requests` library, validating HTTP response codes, response payload fields, and JSON schema keys.

## 1.2 Tools & Technology Stack

Table 1: Automation Technology Stack & Tools

| Category       | Tool / Library       | Specification                     | Engineering Purpose                                          |
|----------------|----------------------|-----------------------------------|--------------------------------------------------------------|
| Language       | Python               | 3.10+                             | Core scripting language and test harness execution           |
| UI Automation  | Selenium WebDriver   | 4.x                               | Browser orchestration, DOM queries, element interaction      |
| Browser Driver | Mozilla GeckoDriver  | System Binary                     | Bridge connecting Selenium WebDriver to Firefox engine       |
| Browser Engine | Mozilla Firefox      | Stable Release                    | Target browser runtime supporting GUI and headless execution |
| API Automation | Requests             | 2.31+                             | Issuing synchronous HTTP REST requests (GET, POST)           |
| Diagnostics    | Python logging       | Standard Library                  | Structured timestamped logging with BDD syntactic tags       |
| Environment    | Linux (Arch / POSIX) | Virtualenv ( <code>.venv</code> ) | Sandboxed execution and dependency management                |

## 2 Target Applications & Endpoints

### 2.1 Web UI Target

- **Application:** The Internet (Herokuapp Practice Portal)
- **Endpoint URL:** <https://the-internet.herokuapp.com/login>
- **DOM Target Elements:** #username, #password, button[type='submit'], dynamic banner alert #flash, and the secure area logout button a.button.secondary.radius.

### 2.2 REST API Target

- **Base Host:** JSONPlaceholder REST API
- **Base URL:** <https://jsonplaceholder.typicode.com>
- **Endpoints Validated:**
  - o GET /posts/1: Retrieval of existing blog entity; asserts HTTP 200, id=1, and payload integrity.
  - o POST /posts: Entity ingestion; asserts HTTP 201 Created, generated id, and transmitted payload fidelity.

## 3 Test Architecture & Design

The automation program is developed with strict modular separation between browser lifecycle factories, UI interaction handlers, API validation components, and execution runners:

Assignment2.py

```
get_firefox_driver()          # Factory: Resolves geckodriver & headless options
class DataDrivenLoginSuite    # UI Suite: Explicit synchronization & assertion hooks
class RestApiTestSuite        # API Suite: BDD-style HTTP requests & JSON asserts
run_ui_data_driven_tests()    # Runner: Iterates through test dataset with screenshot hook
run_api_tests()               # Runner: Executes API verification scenarios
main()                        # Orchestrator: Entry point with guaranteed cleanup
```

## 4 Implementation Details & Key Code

### 4.1 Dynamic Driver Resolution & Factory

The driver initialization routine dynamically locates the geckodriver binary on the host environment via explicit path checking or fallback resolution through `shutil.which`.

```
1 def get_firefox_driver(headless: bool = False) -> webdriver.Firefox:
2     gecko_path = os.getenv("GECKODRIVER_PATH", "/usr/bin/geckodriver")
3
4     if not os.path.exists(gecko_path):
5         gecko_path = shutil.which("geckodriver")
6
7     if not gecko_path:
8         logger.error("geckodriver not found! Install via: sudo pacman -S geckodriver")
9         sys.exit(1)
10
11     logger.info(f"Using geckodriver at: {gecko_path}")
```

```

12     firefox_options = Options()
13     if headless or os.getenv("HEADLESS", "0") == "1":
14         logger.info("Running Firefox in headless mode...")
15         firefox_options.add_argument("--headless")
16
17     firefox_options.add_argument("--width=1920")
18     firefox_options.add_argument("--height=1080")
19
20     service = Service(executable_path=gecko_path)
21     driver = webdriver.Firefox(service=service, options=firefox_options)
22     driver.implicitly_wait(5)
23     return driver

```

Listing 1: Resilient Firefox Driver Factory

## 4.2 Data-Driven UI Login Suite

The `DataDrivenLoginSuite` encapsulates explicit waiting mechanisms on dynamic DOM elements, verifies outcome banners, and enforces session teardown upon successful login.

```

1 class DataDrivenLoginSuite:
2     LOGIN_URL = "https://the-internet.herokuapp.com/login"
3
4     def __init__(self, driver: webdriver.Firefox, timeout: int = 10):
5         self.driver = driver
6         self.wait = WebDriverWait(driver, timeout)
7
8     def navigate_to_login(self):
9         logger.info(f"[GIVEN] Navigating to login portal: {self.LOGIN_URL}")
10        self.driver.get(self.LOGIN_URL)
11
12    def execute_login(self, username: str, password: str):
13        logger.info(f"[WHEN] Entering credentials: username='{username}'")
14        username_field = self.wait.until(EC.visibility_of_element_located((By.ID, "
15        username"))))
16        password_field = self.wait.until(EC.visibility_of_element_located((By.ID, "
17        password"))))
18        submit_button = self.wait.until(EC.element_to_be_clickable((By.CSS_SELECTOR, "
19        button[type='submit']"))))
20
21        username_field.clear()
22        username_field.send_keys(username)
23        password_field.clear()
24        password_field.send_keys(password)
25        submit_button.click()
26
27    def verify_login_outcome(self, expected_status: str) -> bool:
28        logger.info(f"[THEN] Verifying expected outcome: '{expected_status}'")
29        flash_element = self.wait.until(EC.visibility_of_element_located((By.ID, "flash")
30        ))
31        flash_text = flash_element.text
32
33        if expected_status == "success":
34            assert "You logged into a secure area!" in flash_text, f"Expected success
35            message not found. Got: {flash_text}"
36            logout_btn = self.wait.until(EC.element_to_be_clickable((By.CSS_SELECTOR, "a.
37            button.secondary.radius"))))
38            logger.info("        Successfully authenticated to secure dashboard.")
39            logout_btn.click()
40            return True
41        else:

```

```

36         assert "Your username is invalid!" in flash_text or "invalid" in flash_text.
lower(), f"Expected error message not found. Got: {flash_text}"
37         logger.info("        Properly rejected invalid credentials with banner alert."
)
38         return True

```

Listing 2: Data-Driven UI Authentication Suite

### 4.3 REST API BDD Validation Suite

The `RestApiTestSuite` exposes assertion methods conforming to standard BDD syntax for HTTP requests, status codes, field equivalence, and schema key presence.

```

1  class RestApiTestSuite:
2      BASE_URL = "https://jsonplaceholder.typicode.com"
3
4      def __init__(self, timeout: int = 10):
5          self.timeout = timeout
6          self.response = None
7
8      def given_get_endpoint(self, endpoint: str):
9          url = f"{self.BASE_URL}{endpoint}"
10         logger.info(f"[API GIVEN] Sending GET request to: {url}")
11         self.response = requests.get(url, timeout=self.timeout)
12
13     def given_post_endpoint(self, endpoint: str, payload: Dict[str, Any]):
14         url = f"{self.BASE_URL}{endpoint}"
15         headers = {"Content-Type": "application/json; charset=UTF-8"}
16         logger.info(f"[API GIVEN] Sending POST request to: {url}")
17         self.response = requests.post(url, data=json.dumps(payload), headers=headers,
18         timeout=self.timeout)
19
20     def then_status_code_should_be(self, expected_status: int):
21         logger.info(f"[API THEN] Verifying status code is {expected_status}")
22         assert self.response.status_code == expected_status
23
24     def then_json_field_equals(self, field_key: str, expected_val: Any):
25         logger.info(f"[API AND] Checking JSON['{field_key}'] == {expected_val}")
26         assert self.response.json().get(field_key) == expected_val
27
28     def then_json_contains_keys(self, required_keys: List[str]):
29         logger.info(f"[API AND] Checking response contains keys: {required_keys}")
30         for key in required_keys:
31             assert key in self.response.json()

```

Listing 3: REST API BDD Test Suite

## 5 Test Scenarios & Dataset Matrix

### 5.1 UI Data-Driven Test Dataset

Four test permutations were executed against the authentication form:

Table 2: UI Data-Driven Credential Matrix

| Case | Scenario          | Username    | Password             | Expected | Validation Target                    |
|------|-------------------|-------------|----------------------|----------|--------------------------------------|
| 1    | Valid Login       | tomsmith    | SuperSecretPassword! | success  | Dashboard banner & Logout action     |
| 2    | Invalid Username  | invalidUser | SuperSecretPassword! | failure  | Flash error banner: username invalid |
| 3    | Invalid Password  | tomsmith    | WrongPassword123!    | failure  | Flash error banner: password invalid |
| 4    | Non-existent User | qa_tester   | empty_password       | failure  | Flash error banner: username invalid |

Table 3: REST API Scenario Matrix

| ID    | Method | Endpoint | Payload Transmitted                        | Status      | Field Validations                      |
|-------|--------|----------|--------------------------------------------|-------------|----------------------------------------|
| API 1 | GET    | /posts/1 | None                                       | 200 OK      | id=1, userId=1, keys ['title', 'body'] |
| API 2 | POST   | /posts   | {"title": "Arch Linux QA Automation", ...} | 201 Created | title match, userId=99, key ['id']     |

## 5.2 REST API Scenario Matrix

# 6 Console Output & Execution Logs

The script was executed inside a Python virtual environment (.venv). The unedited execution log is detailed below:

```
.venv > python Assignment2.py
(base)
23:00:26 [INFO] Using geckodriver at: /usr/bin/geckodriver
23:00:27 [INFO] =====
23:00:27 [INFO] Starting Assignment 2: Data-Driven UI Automation
23:00:27 [INFO] =====
23:00:27 [INFO]
--- [Iteration 1/4] Valid Credentials ---
23:00:27 [INFO] [GIVEN] Navigating to login portal: https://the-internet.herokuapp.com/login
23:00:34 [INFO] [WHEN] Entering credentials: username='tomsmith'
23:00:35 [INFO] [THEN] Verifying expected outcome: 'success'
23:00:35 [INFO] Successfully authenticated to secure dashboard.
23:00:35 [INFO] [PASS] Iteration 1 passed successfully.
23:00:35 [INFO]
--- [Iteration 2/4] Invalid Username ---
23:00:35 [INFO] [GIVEN] Navigating to login portal: https://the-internet.herokuapp.com/login
23:00:36 [INFO] [WHEN] Entering credentials: username='invalidUser'
23:00:36 [INFO] [THEN] Verifying expected outcome: 'failure'
23:00:36 [INFO] Properly rejected invalid credentials with banner alert.
23:00:36 [INFO] [PASS] Iteration 2 passed successfully.
23:00:36 [INFO]
--- [Iteration 3/4] Invalid Password ---
23:00:36 [INFO] [GIVEN] Navigating to login portal: https://the-internet.herokuapp.com/login
23:00:37 [INFO] [WHEN] Entering credentials: username='tomsmith'
23:00:38 [INFO] [THEN] Verifying expected outcome: 'failure'
23:00:38 [INFO] Properly rejected invalid credentials with banner alert.
23:00:38 [INFO] [PASS] Iteration 3 passed successfully.
23:00:38 [INFO]
--- [Iteration 4/4] Non-existent Account ---
23:00:38 [INFO] [GIVEN] Navigating to login portal: https://the-internet.herokuapp.com/login
23:00:39 [INFO] [WHEN] Entering credentials: username='qa_tester'
23:00:40 [INFO] [THEN] Verifying expected outcome: 'failure'
23:00:40 [INFO] Properly rejected invalid credentials with banner alert.
```

```

23:00:40 [INFO] [PASS] Iteration 4 passed successfully.
23:00:40 [INFO]
=====
23:00:40 [INFO] Starting Assignment 2: REST API BDD Validations
23:00:40 [INFO] =====
23:00:40 [INFO]
--- [API Scenario 1] Validate GET /posts/1 ---
23:00:40 [INFO] [API GIVEN] Sending GET request to: https://jsonplaceholder.typicode.com/posts/1
23:00:43 [INFO] [API THEN] Verifying status code is 200
23:00:43 [INFO] HTTP Status 200 confirmed.
23:00:43 [INFO] [API AND] Checking JSON['id'] == 1
23:00:43 [INFO] Field 'id' matches expected value: 1
23:00:43 [INFO] [API AND] Checking JSON['userId'] == 1
23:00:43 [INFO] Field 'userId' matches expected value: 1
23:00:43 [INFO] [API AND] Checking response contains keys: ['title', 'body']
23:00:43 [INFO] All expected JSON keys are present.
23:00:43 [INFO]
--- [API Scenario 2] Validate POST /posts ---
23:00:43 [INFO] [API GIVEN] Sending POST request to: https://jsonplaceholder.typicode.com/posts
23:00:43 [INFO] Payload: {"title": "Arch Linux QA Automation", "body": "Testing Python
Behave & Selenium framework", "userId": 99}
23:00:54 [INFO] [API THEN] Verifying status code is 201
23:00:54 [INFO] HTTP Status 201 confirmed.
23:00:54 [INFO] [API AND] Checking JSON['title'] == Arch Linux QA Automation
23:00:54 [INFO] Field 'title' matches expected value: Arch Linux QA Automation
23:00:54 [INFO] [API AND] Checking JSON['userId'] == 99
23:00:54 [INFO] Field 'userId' matches expected value: 99
23:00:54 [INFO] [API AND] Checking response contains keys: ['id']
23:00:54 [INFO] All expected JSON keys are present.
23:00:54 [INFO]
=====
23:00:54 [INFO] ASSIGNMENT 2 COMPLETED: All UI & API tests PASSED!
23:00:54 [INFO] =====
23:00:54 [INFO] Cleaning up and closing Firefox session...

```

Listing 4: Full Terminal Execution Log

## 7 Test Results & Verdict Summary

### 7.1 Execution Metrics

Table 4: Test Execution Metrics and Verdict Summary

| Test Suite                     | Total Scenarios | Passed   | Failed   | Duration      | Verdict           |
|--------------------------------|-----------------|----------|----------|---------------|-------------------|
| Data-Driven UI Suite (Firefox) | 4               | 4        | 0        | 13.0 s        | <b>PASSED</b>     |
| REST API BDD Suite (Requests)  | 2               | 2        | 0        | 14.0 s        | <b>PASSED</b>     |
| <b>Combined Suite Total</b>    | <b>6</b>        | <b>6</b> | <b>0</b> | <b>28.0 s</b> | <b>ALL PASSED</b> |

### 7.2 Detailed Result Assessment

- **UI Functional Validation:** All 4 credential pairs evaluated sequentially without session corruption. Valid credentials successfully reached the protected dashboard and invoked the logout teardown. All invalid scenarios verified prompt DOM appearance of negative feedback alert banners.
- **API Contract Compliance:** The GET and POST endpoints returned status codes strictly adhering to REST conventions (200 OK and 201 Created). Payload schema validation verified that keys id, userId, title, and body were present and structurally sound.

## 8 Key Technical Highlights & Best Practices

1. **Zero Fixed Sleep Delays:** By replacing arbitrary delays with `WebDriverWait` and explicit conditions (`visibility_of_element_located`), test execution speed is maximized while eliminating dynamic synchronization race conditions.
2. **Automated Failure Screenshot Capture:** The test runner encapsulates individual test iterations in `try...except` blocks to intercept `AssertionError`, `TimeoutException`, and `WebDriverException`, saving timestamped PNG screenshots into `reports/assignment2_screenshots/` for defect triaging.
3. **BDD-Compliant Telemetry:** Standard Gherkin semantic identifiers (`[GIVEN]`, `[WHEN]`, `[THEN]`, `[AND]`) provide readable execution logs accessible to non-technical stakeholders.
4. **Deterministic Process Cleanup:** A top-level `try...finally` ensures `driver.quit()` is invoked reliably, preventing orphaned background browser processes.

## 9 Conclusion

Assignment 2 demonstrated a scalable, full-stack test automation framework unifying browser UI and RESTful backend validation. The suite achieved a **100% pass rate (6/6 scenarios)** in 28 seconds, establishing reliable patterns for dynamic DOM synchronization, data-driven parameterization, and automated BDD-style assertions.